TACS 2026/27 – Assignment II: Agentic MDE for Embedded Controllers

1. Introduction and Goals

In this second assignment, during 4 weeks, each group uses AI coding agents to specify, build and test a small embedded controller, in a programming language of its choice: software that reads sensors, decides and drives actuators, tested against a simulated world with timed scenarios. In the first assignment, the models were mostly structural, and automated transformations turned them into code. Here they are mostly behavioural (state machines, sequence diagrams and properties over time), and an AI agent turns them into code: the models must be precise enough to guide it, and the tests must be thorough enough to check that it followed them. Code generation (including test code) should be *delegated* as much as possible to AI agents; the writing of other project artifacts (namely requirements specifications, design specifications and test plans in markdown files) may be *assisted* by AI agents under close human control.

The learning goals:

- **Prompt and context engineering**: guide AI agents with good prompts, context files and small tasks.
- **Specify, generate, verify, refine**: specify first, let the agent generate the code, verify it, and refine both.
- **Quality of AI-generated code**: via tests, automated checks and manual reviews against the design, not only for correctness but also maintainability.

Embedded controllers have precise, checkable requirements ("the two traffic lights are never green at the same time"), real concurrency, and failure modes where agents make subtle mistakes. The focus is on the specification and on supervising the agent.

2. The Project in a Nutshell

Each group develops one controller from the pool (Section 3) in short iterations, each covering the following lifecycle activities (with greater emphasis in the first tasks in the first iterations):

- **Specify**: the requirements, the design and the test plan, in markdown files (Sections 5 and 6); agents may help, under your review.
- **Generate**: the agent implements the controller one scenario at a time, test first (Section 6).
- **Verify**: run the scenarios on the real threads and compare each log with its expected log; review the code of each state machine against its diagram.
- **Refine**: when a scenario fails, fix the code (aka have the agent fix the code), or the specification if it was wrong, and repeat.

The controller runs on a normal computer, against simulated sensors and actuators. You need only your language's standard library and a test framework; any other library needs the teachers' approval.

3. Project Pool

The ten problems were assigned at random to the 19 groups (see Groups). Each group works independently and chooses its own language. Every controller has two concurrent components that coordinate their access to a shared resource through a third component, which manages its state or its allocation, as in the worked example. The starting properties are only a start: refine them, and give each time limit (T) a name and a value. Indicate your new repository by November 12 on Groups.

#	Controller	Concurrent components and shared resource	Starting properties
1	Parking lot with entry and exit barriers	a thread per barrier (1 entry, 1 exit); shared: the free-space count	the count is never below zero nor above capacity; a barrier never closes on a car; a car at the entry is let in, or told the lot is full, within T
2	Electric vehicle charging with a shared power limit	two chargers, one thread each; shared: the power budget, granted in fixed steps	total power never above the limit; a waiting charger gets power within T; power is reduced within T when the limit drops
3	Lift bridge over a canal	a road controller (barriers, lights) and a river controller (boat signals), one thread each; shared: the span	the span never moves while a barrier is open or a car is on it; a waiting boat passes within T; the road reopens within T after the boat has passed
4	Road intersection with emergency priority	one thread per approach (north–south, east–west); shared: the crossing area	conflicting directions never green together; a waiting direction gets green within T; an emergency vehicle gets green within T, with amber respected
5	Home alarm with zones and keypads	two keypads, one thread each, with the zones as sensor events; shared: the armed state	arm and disarm from either keypad, with exit and entry delays; the alarm sounds within T of an intrusion; never while disarmed
6	Goods lift between two floors	a controller per floor (call button, landing door), one thread each; shared: the cabin	a landing door opens only when the cabin is at that floor; a call is served within T; alarm if the cabin does not arrive within T
7	Single-lane tunnel with traffic lights at both ends	a controller per end, one thread each; shared: the lane, one direction at a time	never green at both ends; green only once the last car has left; a waiting end gets green within T
8	Railway level crossing with two tracks	a controller per track (approach and exit sensors),	the barriers are down before a train reaches the crossing, and up only

#	Controller	Concurrent components and shared resource	Starting properties
1	Parking lot with entry and exit barriers	a thread per barrier (1 entry, 1 exit); shared: the free-space count	the count is never below zero nor above capacity; a barrier never closes on a car; a car at the entry is let in, or told the lot is full, within T
		one thread each; shared: the road barriers	when no train is near; alarm if a barrier does not confirm within T
9	Car wash with one bay and two entry lanes	a controller per lane (gate, ticket), one thread each; shared: the bay	one car in the bay at a time; first come, first served; the brushes stop within T if a person is detected in the bay
10	Two robots sharing a work zone	a controller per robot, one thread each; shared: the zone	never both robots in the zone; a request for the zone is granted within T; emergency stop within T when the light curtain is broken

Every bounded-response property ("within T") must state the assumptions about the environment, and the faults, under which it holds, as the airlock's[1] P2 does ("unless an alarm comes first"): a charger cannot get power if none is available, nor a robot enter the zone if the other never leaves. **Prefer a small, explicitly simplified model with thorough tests over a realistic one with incomplete verification.**

4. Airlock - Worked Example and Architecture

The teachers provide a worked example, a two-door airlock (not in the pool), in a public repository (https://github.com/joaopascoalfariafeup/airlock), in Rust and in Java: one specification (SPEC.md), one test plan (TEST.md), and a set of scenario files (with test inputs and expected outputs) valid for both implementations in Rust and Java. Its SPEC.md and TEST.md are the templates for yours, and its README.md says which parts are required and which are optional. You may copy and adapt any of it with your agent: record that in the agent log and be able to explain it. Airlock's architecture should be used as a reference architecture in every project:

- **Active objects**: each component has its own operating-system thread and a single input queue (its inbox), bounded and blocking, that receives the messages of all its senders: one queue per component, not a pair of queues per link; a shared resource is owned by one component, the coordinator.
- **Decision functions**: each component decides with a state machine, coded as a function with no I/O, threads or clock, so that it can be reviewed line by line against its diagram.
- **Fake hardware**: the controller talks to the world only through queues, so threads that fake the sensors and actuators stand in for the hardware, and a test driver plays the users.
- **Event log**: every message between components is logged with its time; the tests compare the log with an expected log, using tracecheck, a small tool provided with the example, for any language.

[1] Airlock is the teaching team's working example. See section 4.

- **Time**: the controller uses the real clock, and is given its times when it starts, so the tests run faster (for example, 10 times) by giving shorter times.

Languages: Java 21 or later (JUnit; SpotBugs or Error Prone), C# (.NET 8 or later; xUnit or NUnit; warnings as errors), Python 3.12 or later (pytest, ruff, mypy), Rust (cargo test, clippy), or C++20 (GoogleTest or Catch2, clang-tidy, ThreadSanitizer); C only on request. Use dedicated threads, not async tasks or virtual threads, and a bounded blocking queue with a timeout (ArrayBlockingQueue, BlockingCollection, queue.Queue, sync_channel; in C++, a small one with a mutex and a condition variable). In Python, threads take turns, so allow a wider timing tolerance. On Windows, WSL is recommended for Rust and C++.

5. Deliverables

Everything is delivered in a GitHub repository created by the group for this assignment (not the one of the first assignment). Invite the teachers (@jacomecunha, @joaopascoalfariafeup, @lessthelonely) and put its link in the Groups spreadsheet **by November 12**. Each student's contribution is assessed through the substance of their commits, pull requests, reviews and agent log entries, and in the oral discussion. At the end of week 4 (**by December 9**), the repository contains, laid out as in the worked example:

- **SPEC.md**: the requirements and the design, about 5 to 10 pages with diagrams and tables (Section 6).
- **TEST.md**: the test plan: the criteria, the scenarios, and how the tests are built (Section 6).
- **Code and tests**: the controller, the test driver, and the scenario files with their expected logs; a README.md says how to run the one command that runs every check.
- **Agent files**: the agent context file and the agent log (Section 7).
- **Retrospective** (RETROSPECTIVE.md, short): what worked well and what did not, what you would improve, how the collaboration between people and agents worked, the agent's mistakes and how they were caught, and a personal reflection by each student.

In the last week, each group demonstrates its controller, followed by the oral discussion.

6. Development Process

Phase 1 – Specify

Before any controller code, write the first version of SPEC.md and TEST.md, following the worked example. Give every item an id, and cite the ids in the documents, the code (// Implements: P1) and the tests (// Verifies: P1).

- **SPEC.md:**
  - **Context**: what the controller does; its sensors, actuators and users.
  - **Functional requirements**: the users' journeys (J1, J2, …, a sentence each), and the alternative and exception flows that can happen in them (A1, E1, …), each with what the controller does.
  - **Non-functional requirements**: temporal properties (P1, P2, …), each one sentence that can be checked on a log, with the hazard it guards against, and its times as named values ("within RESPONSE_LIMIT = 11 s"). Ask of every journey what must never happen (safety) and what must happen within a time limit (bounded response), and cover faults and the shared resource too.

    - **Constraints, assumptions and exclusions**: on the solution, on the environment, and what you leave out.
    - **Design**: a structural diagram (communication, component or class diagram), a state machine for each concurrent component with its transitions numbered, and a sequence diagram of the main scenario, all as text (Mermaid or PlantUML); a table of the messages; how the shared resource is requested, granted and released; and the main design decisions (D1, D2, …), each with its rationale.
- **TEST.md: Test plan** with the criteria that show the tests are good enough (Section 6); the scenarios (S1, S2, …), in the order you will implement them, each with what happens, its inputs, its expected log and the transitions it adds; and how the tests are built.
- **Agent context file** (AGENTS.md, .github/copilot-instructions.md or CLAUDE.md): the specification as reference, the architecture, the conventions, test-first work, and what is forbidden: unbounded queues, state shared between components other than the log and the run context (clock, stop flag), libraries without approval, and risky constructs (unwrap() in Rust, a bare except: in Python, …).

The specification is a living document: update it whenever verification shows it was wrong or incomplete, or as you extend your requirements.

Phase 2 – Generate, Verify, Refine

Test first. Once the minimum test infrastructure is in place (a skeleton of the system, the log and the test driver), implement one scenario at a time, in the order of the test plan: (1) add the scenario file and its expected log, and see the test fail; (2) let the agent implement just enough to pass, without changing the expected log; (3) refactor, with all the scenarios as a safety net. You command the specification; the agent generates the code and may write the scenario files; you review how the specification maps to the code.

- **Scenarios and expected logs**: a scenario is a text file of timed inputs and faults; its expected log lists the messages the system must log, in order, with time bounds where a requirement sets a time. Expected logs are written from the specification, never from the program's output. One test runs every scenario and has tracecheck compare each actual log with its expected log. Start simple: a timed script of inputs is enough, with one complete expected log, for the main scenario, and only the end of the others.
- **Good enough when**: the scenarios cover all the journeys and flows in the requirements, and all states and transitions in every state machine; every temporal property defined in the requirements is challenged by a scenario and adequately checked from the event log; when multiple outcomes are possible due to concurrency, multiple runs should be executed; the code of each state machine is reviewed against its diagram (design conformance), including events that should be ignored.
- **One command** (mvn test, dotnet test, pytest, cargo test or ctest) runs every test, and the linter reports no warnings. A scenario that fails unexpectedly is reduced and kept as a regression test.
- **Code review**: review every change the agent makes before merging it, with a short checklist (timing edge cases, unbounded queues, timers not cancelled, error handling).

7. Use of AI Agents and Teamwork

Use AI agents for most of the code: GitHub Copilot in agent mode is the default; others, such as Claude Code or Cursor, are allowed. You may reuse or write agent skills (the worked example has three, in .github/skills/), and record in the agent log which skill each task used. You are responsible for everything you submit: every member must be able to explain the properties, the scenarios and their expected logs, and the decision logic in depth, and the threads and queues at concept level.

- **Independent oracles**: you write the properties; the expected logs are written by you, or drafted by an agent session that has not seen the code, and reviewed by you. If one session writes both the code and its tests, they share its mistakes. When a test fails, check it against the specification first.
- **Agent log**: for each significant task, the tool and model, the goal, the context, a summary of the prompt, what was accepted, and what was wrong and how you noticed. Each student records at least two agent changes they rejected or corrected.
- **One step at a time**: context, requirements, design, test plan, then code; the agent proposes, you decide, and you review each step before the next iteration.
- **Team**: the specification, test plan and agent context file are your contract: change them through reviewed pull requests. Split the work between the oracles (properties, scenarios, expected logs), the controller and the test driver, and rotate the roles at least once. Work in small branches whose pull requests cite the ids they serve, are reviewed by another member, and pass the one command in CI.
- **Reports** are written by you, short and personal.

8. Suggested Schedule

Week	Focus	Deliverables (end of week)
1	The assignment and the problems; walkthrough and demonstration of the worked example; specification	First version of the specification (Section 6, still independent of the language), choice of language, agent context file, first agent log entries, and the worked example installed and run.
2	Generate and verify the main behaviour	The main scenario passing, test first, with its expected log; one command running all the tests
3	Concurrency, faults and refinement	The remaining scenarios, test first (timers, faults, stress scenarios), fixes for the failures found, regression tests
4	Quality and wrap-up	Final reviews and documentation, agent log, retrospective (Sections 5 and 7), demonstration

9. Evaluation

Component	Weight
Requirements specification quality	10%

Component	Weight
Design specification quality	10%
Test specification quality	10%
Working solution, consistent with specs, passing the tests	20%
Agent context files and logs	10%
Retrospective	15%
Oral discussion and weekly progress	25%

The first four components assess the product, and the last three the process, 50% each. Individual marks are adjusted according to each student's contributions and the final discussion.

10. Resources

- **The worked example** (https://github.com/joaopascoalfariafeup/airlock), with a guided walkthrough in the first week.
- **On working with agents** (recommended, about two hours): K. Tantithamthavorn, Agentic Software Engineering: A Practical Guide for the AI-Native Engineer, Monash University, 2026, https://book.agentic-swe.dev: Chapter 6, sections 7.1 and 7.2, and Tutorial 6.
- **Optional, on agents**: M. Pocock, "Software Fundamentals Matter More Than Ever" (https://www.youtube.com/watch?v=v4F1gFy-hqg); https://agents.md; on martinfowler.com, K. Morris, "Humans and Agents in Software Engineering Loops", and B. Böckeler, "Context Engineering for Coding Agents" and "TDD inside the agent loop: theater or actual value?" (2026).
- **Optional, on the domain**: E. A. Lee and S. A. Seshia, Introduction to Embedded Systems, 2nd ed., free at leeseshia.org (chapters 3, 5 and 13); D. Harel, "Statecharts" (1987); R. V. Binder, Testing Object-Oriented Systems (1999), on testing state machines; for Rust, The Rust Programming Language, chapters 1 to 10 and 16.